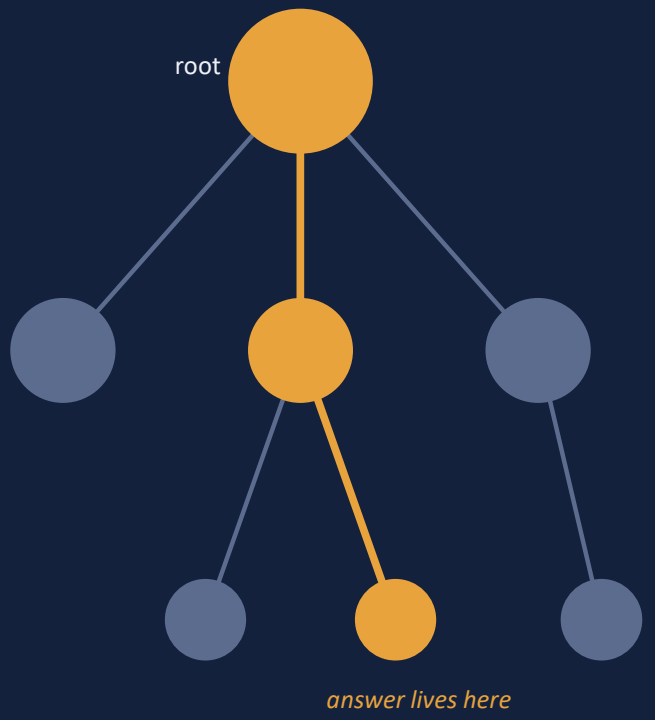
Vectorless RAG

From similarity search to reasoning-based retrieval — a step-by-step concept build with seven hands-on labs



Python labs · works with any LLM API · ~5 hours end to end

MODULE 1

Why retrieval at all?

What problem RAG solves — and why retrieval quality caps answer quality



Three problems RAG solves

1 Knowledge cutoff

Model weights freeze at training time. Last quarter's results or yesterday's policy change simply aren't in there.

2 Private data

Your contracts, syllabus, manuals and tickets were never in the training data. The model cannot know them.

3 Context limits & cost

Even huge context windows are slow and costly to fill on every query, and models use very long inputs unevenly.

RAG in one sentence: fetch only the relevant evidence at question time, put it in the prompt, and let the LLM answer from that evidence — with citations.

The universal RAG loop

1 Question

The user asks something in natural language

→

2 Retrieve

Find the evidence in your documents

→

3 Augment

Insert that evidence into the prompt

→

4 Generate

LLM answers, grounded in evidence, with citations

this is where every RAG variant differs

Key insight

Vector RAG, vectorless RAG, hybrid RAG and graph RAG are not different systems end to end. They are different answers to one question: how do we FIND the right evidence? Steps 1, 3 and 4 stay the same in every lab you will build.

Retrieval caps answer quality

Answer quality

≤

Retrieval quality

A great LLM cannot recover evidence it never saw

1	**Miss**	Retrieval misses the evidence → the LLM guesses or says 'not found'. Hallucination risk.
2	**Partial**	Retrieval finds half the evidence → a confident but incomplete answer. The most dangerous case.
3	**Noise**	Retrieval returns lots of loosely related text → the model gets distracted and cost rises.

So the real question of this deck: what is the best way to FIND the right evidence?

MODULE 2

Vector RAG: how it works, where it breaks

Understand the default approach deeply before replacing it



The vector RAG pipeline

OFFLINE

once per corpus

Load

Read PDFs, pages, HTML

→

Chunk

Cut into ~500–1000 token pieces with overlap

→

Embed

Each chunk → a vector of numbers

→

Store

Save vectors in a vector database

ONLINE

every query

Embed query

Question → a vector

→

Search

Find the top-K nearest chunk vectors

→

Stuff

Put those K chunks in the prompt

→

Generate

LLM writes the answer

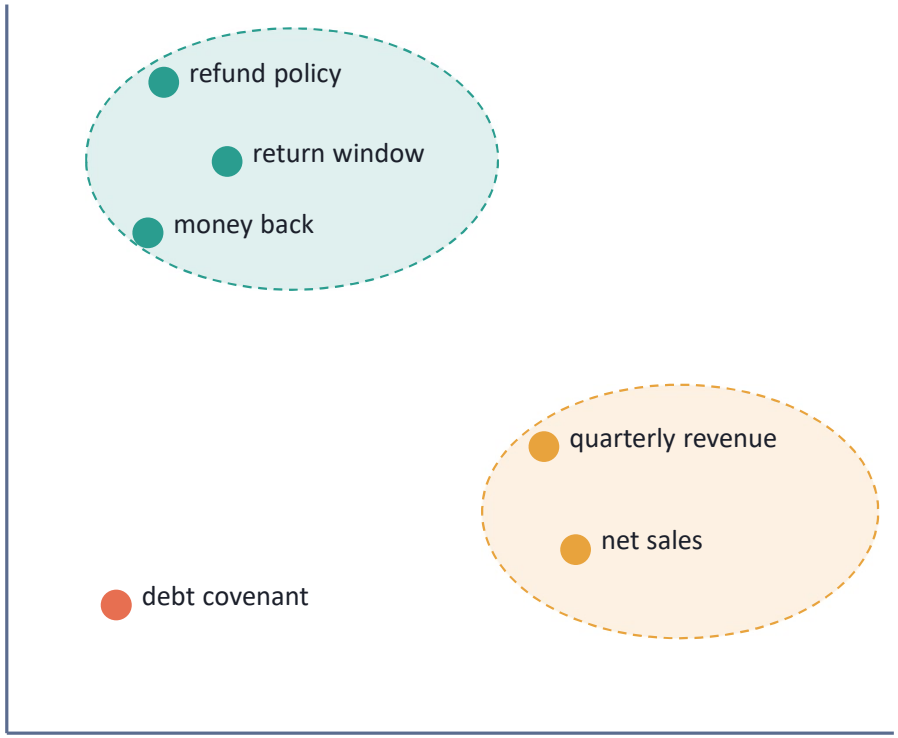
Hidden assumption: the answer lives inside ONE chunk whose meaning is close to the question's meaning. Remember this — Module 3 challenges it.

Embeddings in one slide

- An embedding model maps text to a vector, typically 384–3,072 numbers
- Texts with similar meaning land close together
- Closeness is measured with cosine similarity
- Retrieval = return the K chunks closest to the question

$$\cos(a, b) = (a \cdot b) / (|a| \times |b|)$$

1.0 = same direction (same meaning)
0.0 = unrelated



2-D cartoon — real embeddings have hundreds of dimensions

Where vector RAG shines

✓ Paraphrase tolerance

"How do I get my money back?" finds the "Refund policy" section even with zero shared keywords.

✓ Massive scale

Approximate nearest-neighbour search over millions of chunks returns in milliseconds.

✓ Cheap per query

One embedding call plus one LLM call. Easy to serve at high traffic.

✓ No structure needed

Works on chats, tickets, FAQs, emails and reviews — text with no table of contents.

Six ways vector RAG breaks

1 Chunk boundaries cut meaning

A table's header lands in chunk 41 and its rows in chunk 42. Neither makes sense alone.

2 Similar ≠ relevant

"Debt levels declined" commentary outscores the actual debt table that holds the answer.

3 Structure is lost

A chunk no longer knows it belongs to Financial statements › Notes › Note 7.

4 Cross-references break

"See Note 7" has no embedding link to Note 7, so the reference is never followed.

5 Top-K blind spot

The evidence is spread across 9 places; you retrieved only 5 of them.

6 Opaque retrieval

A cosine score of 0.82 does not explain WHY a chunk was chosen. Hard to audit.

Worked failure: a 10-K debt question

Question What was total long-term debt at fiscal year-end, and what is the interest rate on the largest note?

What vector RAG retrieves (top-3)

0.84	**MD&A, p.42**	*"We continued to reduce our debt levels during the year…"*
0.81	**Risk factors, p.18**	*"Our indebtedness could adversely affect our flexibility…"*
0.79	**MD&A, p.44**	*"Liquidity remains strong; see Note 7 for details."*
0.62	**Note 7, p.89 — rank 9**	*"4.25% Senior notes due 2031 … 1,250"* ← *NOT retrieved*

Illustrative example — the pattern is common on real filings

Why it failed

- "Debt" appears everywhere, so narrative text wins on similarity
- The answer is a numeric table that looks nothing like the question
- "See Note 7" was retrieved, but nothing follows the pointer
- Result: a vague answer, or a wrong number

MODULE 3

Vectorless RAG: retrieval as reasoning

Build a tree index of the document, then let the LLM navigate it like an expert



How an expert reads a 300-page report

	1	2	3	4
Human expert	Opens the table of contents	Thinks: debt figures live in the financial statements	Jumps to Notes → Note 7 Debt	Reads pages 88–90 in full and answers
Vectorless RAG	LLM reads the tree index: titles + summaries	LLM selects a branch and writes down why	Drills down level by level to a leaf section	Fetches the full section text → answer with page citation

No chunking. No embeddings. No vector database. Retrieval becomes navigation plus reasoning over the document's own structure.

“Vectorless” means more than one thing

A Reasoning over a tree index

An LLM navigates a table-of-contents-like hierarchy (PageIndex style). The focus of this deck.

B Lexical search

BM25, Postgres full-text, Elasticsearch. Great for exact keywords, IDs, codes and numbers.

C Structured retrieval

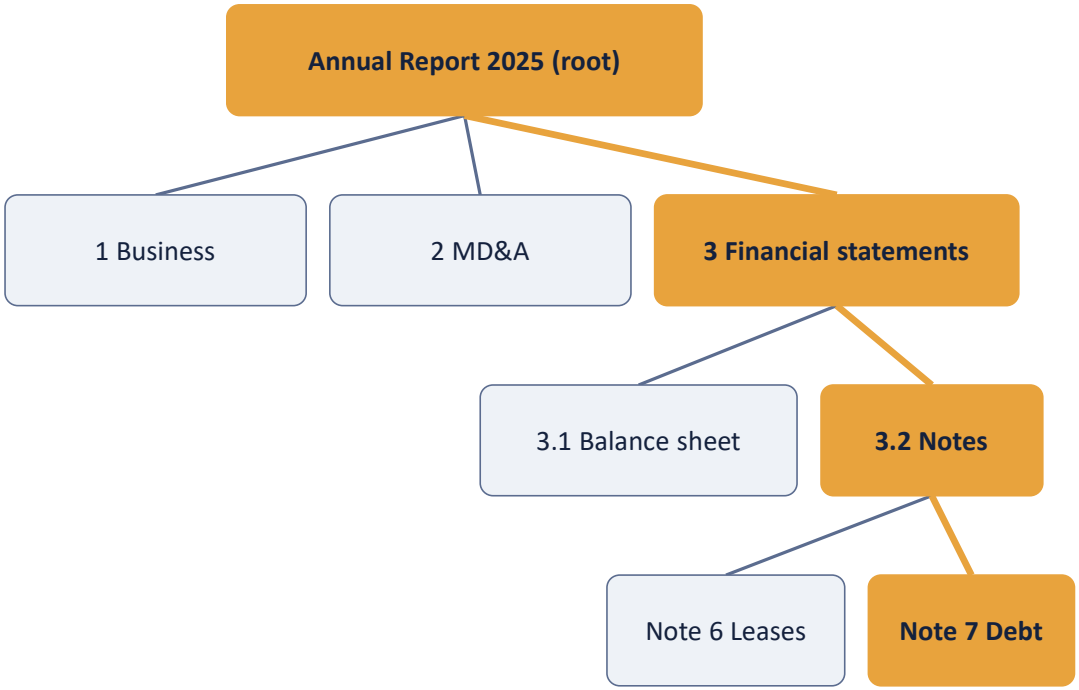
Text-to-SQL or knowledge graphs, for data that is already tabular or relational.

D Long-context / no retrieval

If the document fits in the context window, just send it (with prompt caching).

They are tools, not rivals — Module 5 combines them into a hybrid.

Anatomy of the tree index



Highlighted = path the LLM walks for a debt question

```
{
  "node_id": "3.2.7",
  "title": "Note 7 – Long-term debt",
  "start_page": 88,
  "end_page": 90,
  "summary": "Senior notes by maturity
    and coupon; total debt;
    covenants; repayment table.",
  "children": []
}
```

LLM navigates using titles + summaries (cheap). Full text is fetched only for the chosen leaves.

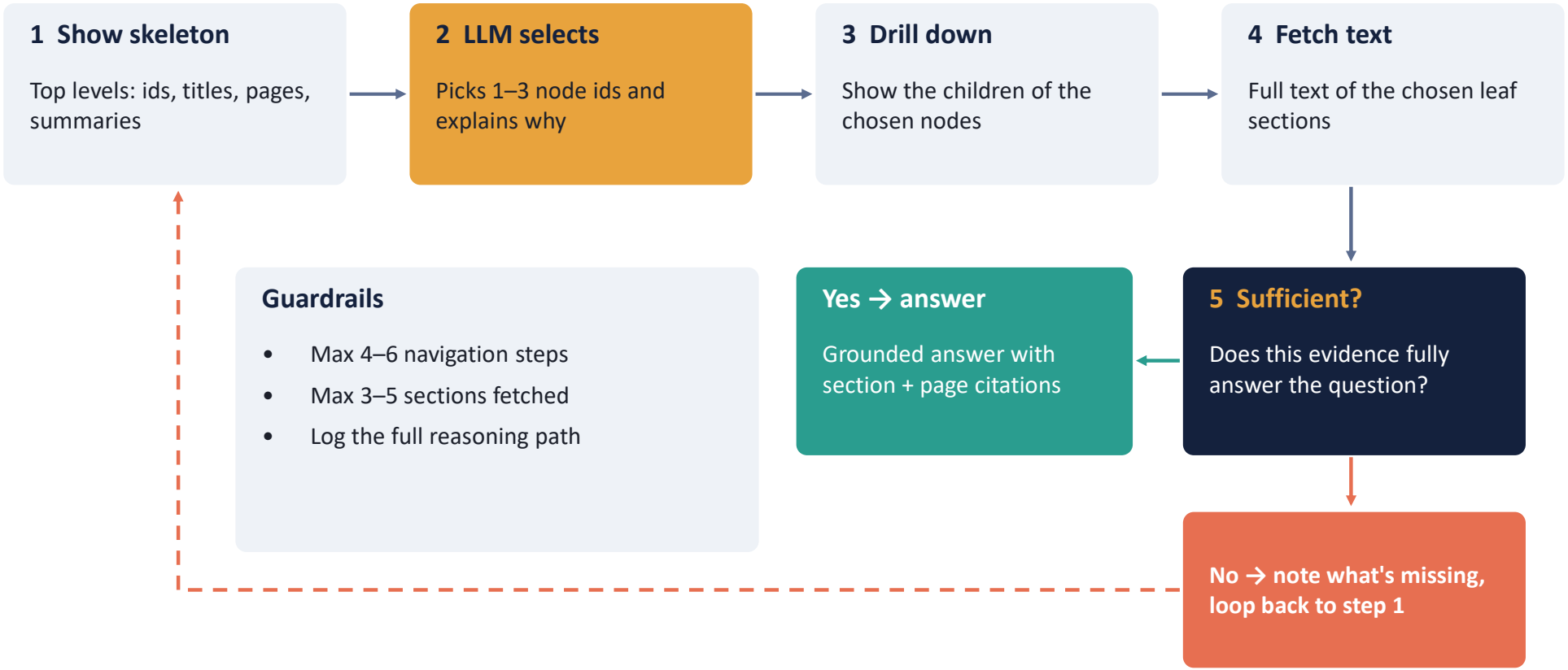
Phase 1 — Indexing, step by step

1	**Parse**	Extract text page by page and KEEP page numbers. OCR-scanned pages; extract tables carefully.
2	**Detect structure**	Use the PDF outline/TOC if present → else heading heuristics (font size, "3.2" numbering) → else ask the LLM.
3	**Build the tree**	Nest sections as parent/child, assign node ids and page ranges, split leaves longer than ~10 pages.
4	**Summarize bottom-up**	Leaves from their text, parents from their children's summaries. Name entities, numbers, tables.
5	**Persist**	Save the tree as JSON beside the page texts. Re-index only the sections that change.

Cost is paid once per document

- About one LLM call per node to write summaries
- The same tree serves every future query
- Only changed sections need re-indexing

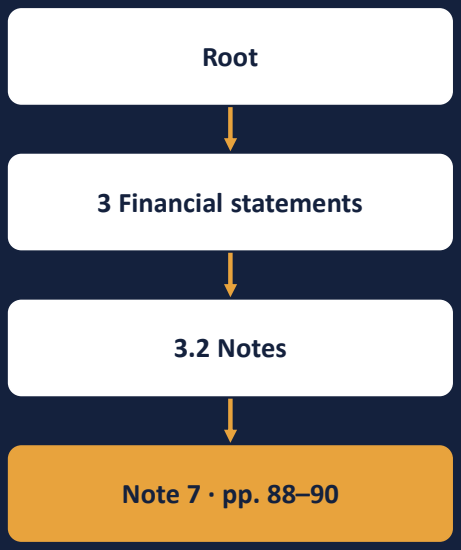
Phase 2 — Retrieval as a reasoning loop



Worked success: the same 10-K question

1	**Sees root's children**	*"Debt amounts and rates live in the financial statements' notes, not in MD&A narrative."*	→ 3 Financial statements
2	**Sees 3.1 … 3.4**	*"A debt footnote will list each note's coupon; the balance sheet holds the total."*	→ 3.2 Notes
3	**Sees Notes 1–14**	*"Note 7 covers long-term debt. Also fetch 3.1 to cross-check the total."*	→ 3.2.7 + 3.1
4	**Reads pp. 88–90, 61**	*Sufficient. Answer: total debt and the largest note's coupon, cited.*	→ Note 7 p.89; BS p.61

The path is the explanation



Vector vs vectorless at a glance

Dimension	Vector RAG	Vectorless RAG
Retrieval unit	Fixed-size chunks	Natural sections (tree nodes)
How relevance is judged	Embedding similarity	LLM reasoning over titles + summaries
Infrastructure	Embedding model + vector DB	LLM + JSON tree (any storage)
Document structure	Discarded	Preserved and used
Cross-references	Not followed	Can be followed ("see Note 7")
Explainability	Similarity scores	Readable path + page citations
Query latency	Low — milliseconds to search	Higher — several LLM calls
Cost per query	Low	Higher
Corpus scale	Millions of chunks	A few long documents per query
Best for	Fuzzy questions, many short texts	Precise questions, long structured docs

MODULE 5

Is vectorless RAG always better?

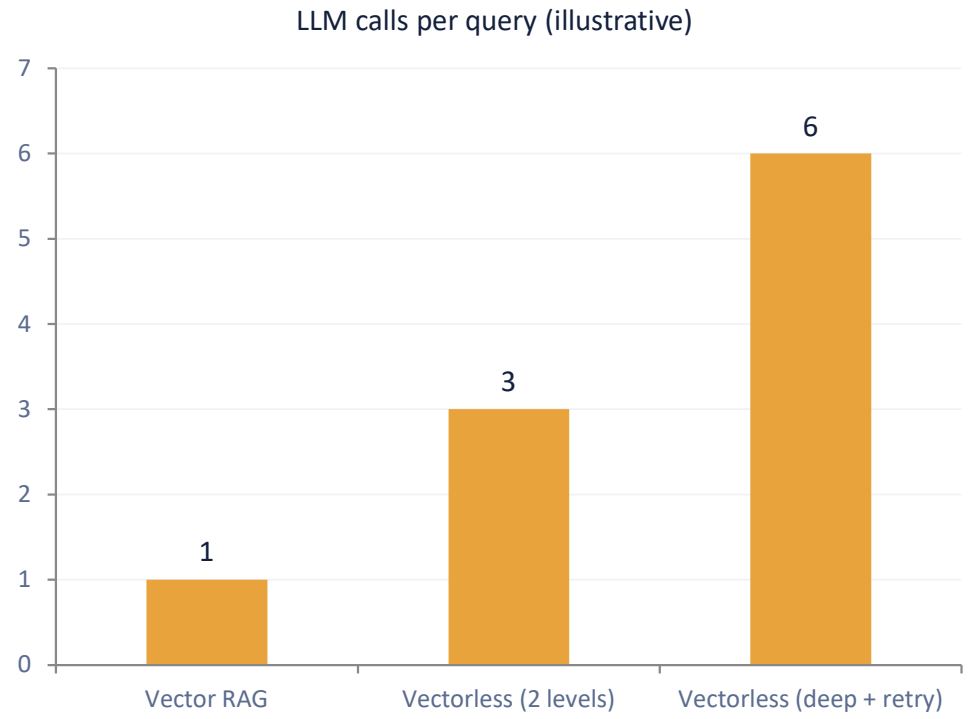
Short answer: no. It is better at a specific, important job — and worse at others



Short answer: no — here is the trade

Vectorless trades query-time compute for retrieval precision.

- Each tree step is an LLM call: seconds, not milliseconds
- Cost grows with traffic × tree depth × retries
- Errors compound: one wrong branch early and the evidence is never seen
- Indexing a document costs LLM calls too (paid once)



Illustrative — your numbers come from Lab 6

Where vectorless RAG loses

✗ Huge, flat corpora

100,000 support tickets have no table of contents. A tree over them means nothing.

✗ Many short documents

FAQs, chats and reviews: structure adds nothing, and similarity works well.

✗ Vague, exploratory queries

"Anything about customer frustration?" is a similarity question, not a navigation one.

✗ Tight latency budgets

Autocomplete, voice assistants and high-traffic search need answers in under a second.

✗ Poorly structured documents

Scanned or flat PDFs with weak headings produce weak trees and weak summaries.

✗ Navigation errors

Vague summaries or a weak model send the search down the wrong branch.

Where vectorless RAG wins

✓ Long, structured documents

10-Ks, contracts, regulations, manuals and textbooks with a real table of contents.

✓ Precision & numbers

Tables and figures that look nothing like the question but sit in a predictable section.

✓ Cross-references

"See Note 7" and multi-section answers that need several parts of one document.

✓ Auditability

A readable reasoning path plus page citations — valuable in regulated domains.

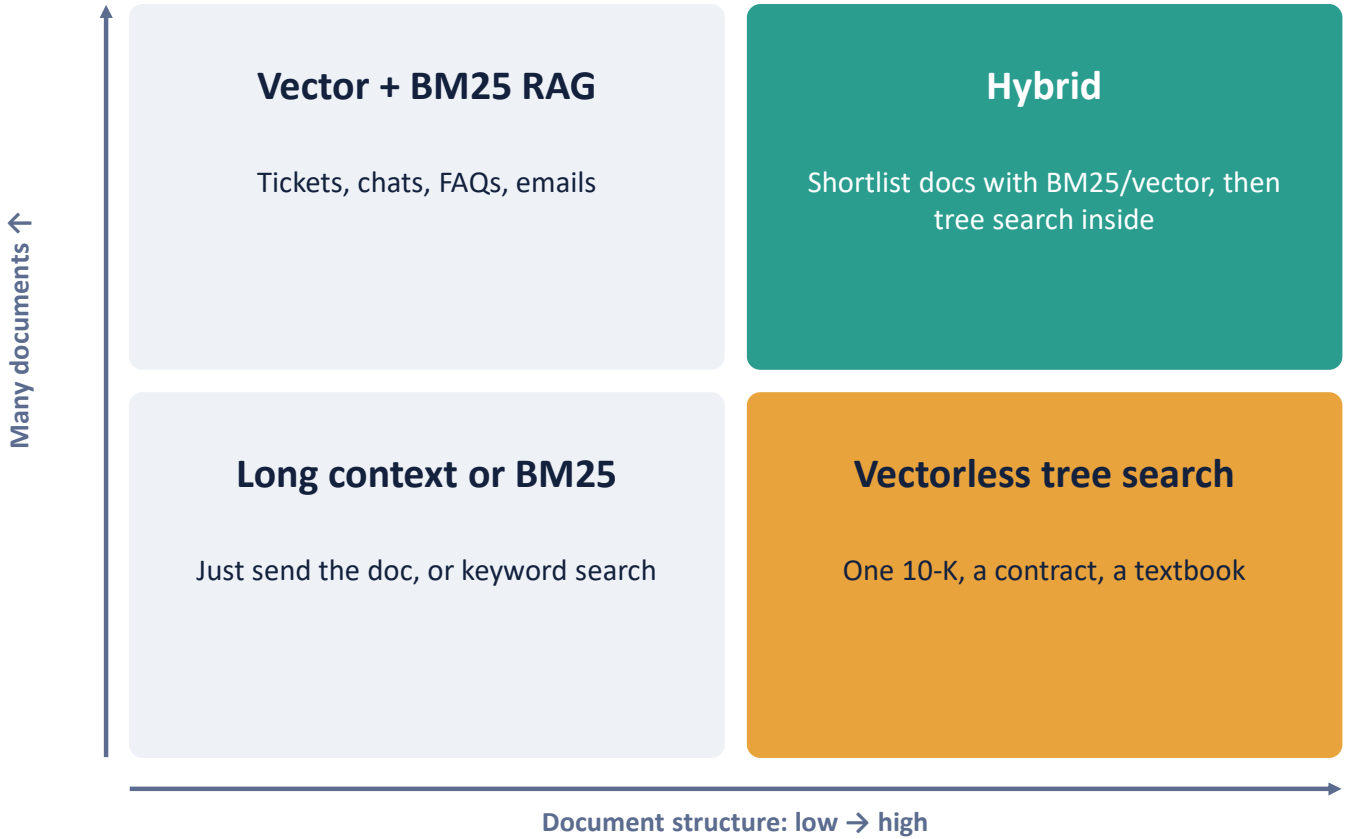
✓ Lighter infrastructure

No embedding model or vector database to run, sync or re-embed.

✓ Curriculum content

Syllabus-ordered textbooks: chapter → topic → subtopic maps cleanly to a tree.

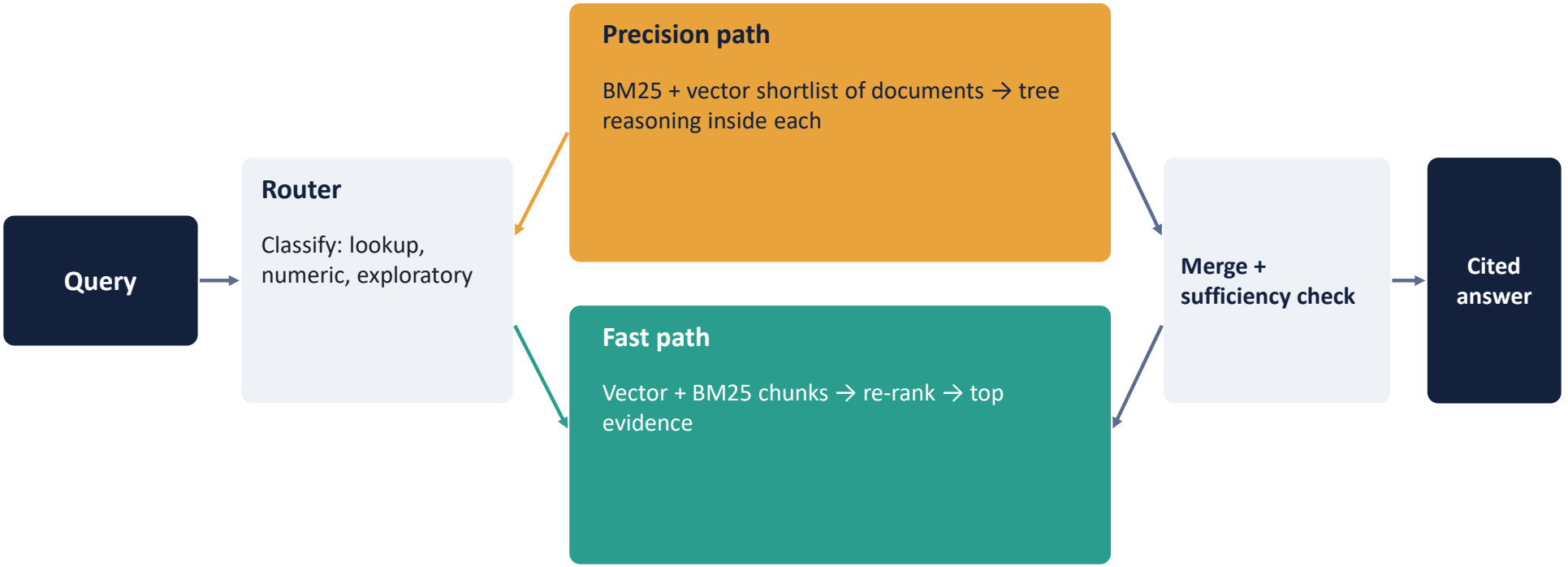
Decision guide: pick by your corpus



Override rules

- Need answers in under 1 s → vector / BM25
- Need an audit trail → add tree search
- Exact IDs, codes, SKUs → always include BM25
- Doc fits in context and traffic is low → just send it

The pragmatic answer: hybrid



Make it fast: cache tree skeletons with prompt caching, cache popular paths, and cap navigation steps.

Reading benchmark claims critically

98.7%

FinanceBench accuracy reported by VectifyAI for its PageIndex-based system

Impressive — and worth reading carefully

1. **Vendor-reported**
   Published by the framework's authors, not an independent evaluation.

2. **One domain**
   Financial filings are long and highly structured: the ideal case for tree navigation.

3. **Baseline matters**
   Vector baselines are often naive: no hybrid search, re-ranking or metadata filters.

4. **Your data decides**
   Run Lab 6 on your documents, questions and latency budget.

MODULE 6

Production & wrap-up

What changes when real users and real documents arrive



Production checklist

Indexing

- Version each tree by document hash
- Re-index only changed sections
- Validate trees: every page covered, no overlaps
- OCR + table extraction for scanned or numeric docs

Query time

- Guardrails: max depth, max nodes, timeouts
- Prompt-cache tree skeletons; cache popular paths
- Fall back to BM25/vector if tree search finds nothing
- Use a small model to navigate, a strong one to answer

Quality

- Log every traversal path and review misses weekly
- Golden eval set per document type
- Re-run evals on every prompt or model change
- Track latency and cost per query bucket

Key takeaways

1. Answer quality is capped by retrieval quality — fix retrieval first.
2. Vector RAG finds what SOUNDS similar; vectorless RAG reasons about where the answer LIVES.
3. Vectorless shines on long, structured, precision-critical documents, and its path is auditable.
4. It costs more latency and LLM calls per query, and struggles on flat, huge or vague corpora.
5. **Not always better. Measure on your own data — in production, hybrids usually win.**

Checkpoint quiz

Q1 Why can a chunk with cosine score 0.84 be worse than one scoring 0.62?

Q2 Which two fields of each node does the LLM navigate by — and why not full text?

Q3 Your corpus is 200,000 customer chat logs. Which approach, and why?

Q4 Name two guardrails you would add to the tree-search loop.

Q5 A 400-page regulation with strict audit needs: design the retrieval.

Answers are in the speaker notes

Try each one before you look. If you get stuck, revisit Modules 2, 3 and 5.

Retrieval is a reasoning problem

…when your documents have structure. Build both. Measure both. Combine them.

Further reading

- PageIndex — VectifyAI (open source, GitHub)
- FinanceBench — Islam et al., 2023
- Lost in the Middle — Liu et al., 2023
- BM25 — the rank_bm25 Python package
- Anthropic — Contextual Retrieval (hybrid BM25 + embeddings)